VOLUME II · ALL 23 EXERCISES SOLVED · MOCK PAPER · WALK-IN SHEET

CMS 708 — Every Practice Exercise, Worked

The course document sets 23 practice exercises across Modules 2, 3 and 4 and gives no solutions. This volume is all 23, written out as complete compilable programs, plus the supplied past-style questions worked in full, trace drills, error-spotting drills and a mock paper.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Monday 10 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturer:** the lecturer

HOW TO USE THIS PACK

Do not read the solutions first. Cover the code, write the program on paper, then compare. Programming is the one subject where reading the answer feels identical to knowing it and is not.

TIME YOU HAVE	DO THIS
3 hours	Write §2 exercises 4, 7, 12, 15, 18, 21 and 23 **by hand on paper** → check → then sit the mock paper (§6).
90 minutes	The seven exercises above, then the trace drills (§3) and error drills (§4).
45 minutes	Skim every solution for its **shape**, then §3, §4 and the worked questions in §5.
10 minutes at the door	§7 only — the skeletons.

1 The four skeletons that answer almost any question

A · read → calculate → print

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    cout << "Enter two numbers: ";
    cin  >> a >> b;
    cout << "Result: " << a + b << endl;
    return 0;
}
```

B · read → decide → print

```
#include <iostream>
using namespace std;

int main() {
    int x;
    cout << "Enter value: ";
    cin  >> x;
    if (x >= 18) cout << "Yes" << endl;
    else         cout << "No"  << endl;
    return 0;
}
```

C · loop a known number of times

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {
        cout << i << endl;
    }
    return 0;
}
```

D · function + main

```
#include <iostream>
using namespace std;

double compute(double x) {      // definition
    return x * x;
}

int main() {
    cout << compute(5) << endl;  // invocation
    return 0;
}
```

Every one of the 23 exercises is skeleton **A, B, C or D**, or two of them combined. Recognise which before writing a line.

Module 2 — C++ fundamentals

1 · STUDENT NAME, AGE AND GPA IN A SENTENCE

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name = "Ada";
    int    age  = 20;
    double gpa  = 3.5;

    cout << "Student " << name << " is " << age
         << " years old with a GPA of " << gpa << "." << endl;
    return 0;
}
```

2 · ONE VARIABLE OF EACH DATA TYPE

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int    count  = 42;
    double price  = 19.99;
    char   grade  = 'A';
    bool   passed = true;
    string title  = "Structured Programming";

    cout << "int:    " << count  << endl;
    cout << "double: " << price  << endl;
    cout << "char:   " << grade  << endl;
    cout << "bool:   " << passed << endl;    // prints 1
    cout << "string: " << title  << endl;
    return 0;
}
```

Note: a `bool` prints as **1** or **0**, not "true"/"false".

3 · FAVOURITE COLOUR AND FOOD

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string colour, food;

    cout << "Enter your favourite colour: ";
    cin  >> colour;
    cout << "Enter your favourite food: ";
    cin  >> food;

    cout << "Your favourite colour is " << colour
         << " and your favourite food is " << food << "." << endl;
    return 0;
}
```

4 · SUM, DIFFERENCE, PRODUCT AND QUOTIENT HIGH YIELD

```
#include <iostream>
using namespace std;

int main() {
    double a, b;
    cout << "Enter two numbers: ";
    cin  >> a >> b;

    cout << "Sum: "        << a + b << endl;
    cout << "Difference: " << a - b << endl;
    cout << "Product: "    << a * b << endl;

    if (b != 0)
        cout << "Quotient: " << a / b << endl;
    else
        cout << "Cannot divide by zero." << endl;
    return 0;
}
```

Two marks hide here: declaring the variables `double` so the quotient is not truncated, and **guarding against division by zero**.

5 · VOTING ELIGIBILITY

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin  >> age;

    if (age >= 18)
        cout << "You can vote" << endl;
    else
        cout << "You are too young to vote." << endl;
    return 0;
}
```

6 · AREA OF A RECTANGLE

```
#include <iostream>
using namespace std;

int main() {
    double length, width, area;

    cout << "Enter length: ";
    cin  >> length;
    cout << "Enter width: ";
    cin  >> width;

    area = length * width;
    cout << "Area of the rectangle: " << area << endl;
    return 0;
}
```

7 · EVEN OR ODD USING `%` HIGH YIELD

```
#include <iostream>
using namespace std;

int main() {
    int number;
    cout << "Enter a number: ";
    cin  >> number;

    if (number % 2 == 0)
        cout << number << " is even." << endl;
    else
        cout << number << " is odd."  << endl;
    return 0;
}
```

The modulus idiom is worth memorising as a family: `n % 2 == 0` even · `n % 2 != 0` odd · `n % 5 == 0` divisible by 5 · `n % 100` the last two digits · `n / 10 % 10` the tens digit. Almost every "check whether…" number question is one of these.

THE PATTERN BEHIND MODULE 2

Every one of these seven is skeleton **A** or **B**: declare with the right type → prompt → `cin` → compute or compare → `cout` with a label. If a question in this style appears, write the skeleton first and fill the middle.

8 · TEMPERATURE: COLD, WARM OR HOT

```
#include <iostream>
using namespace std;

int main() {
    double temp;
    cout << "Enter the temperature (°C): ";
    cin  >> temp;

    if      (temp < 15) cout << "It is cold." << endl;
    else if (temp < 30) cout << "It is warm." << endl;
    else                cout << "It is hot."  << endl;
    return 0;
}
```

Why `else if` and not three separate `if`s: the chain guarantees exactly one branch runs, and each later test already knows the earlier ones failed — so `temp < 30` means "between 15 and 30" without writing both bounds.

9 · ATM MENU WITH `SWITCH`

```
#include <iostream>
using namespace std;

int main() {
    int choice;

    cout << "ATM MENU\n";
    cout << "1. Check Balance\n2. Deposit\n3. Withdraw\n";
    cout << "Enter choice: ";
    cin  >> choice;

    switch (choice) {
        case 1:
            cout << "Your balance is N50,000." << endl;
            break;
        case 2:
            cout << "Deposit successful."     << endl;
            break;
        case 3:
            cout << "Withdrawal successful."  << endl;
            break;
        default:
            cout << "Invalid choice."         << endl;
    }
    return 0;
}
```

10 · FIRST 10 SQUARE NUMBERS

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 10; i++) {
        cout << i << " squared = " << i * i << endl;
    }
    return 0;
}
```

11 · PASSWORD LOOP UNTIL CORRECT

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string password;

    cout << "Enter password: ";
    cin  >> password;

    while (password != "secret") {
        cout << "Wrong password! Try again: ";
        cin  >> password;
    }
    cout << "Access granted!" << endl;
    return 0;
}
```

13 · MULTIPLICATION TABLE 1 TO 5, NESTED LOOPS

```
#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            cout << i << " x " << j << " = " << i * j << endl;
        }
        cout << endl;          // blank line after each table
    }
    return 0;
}
```

Prints **25 lines** in five blocks. Outer loop = which table; inner loop = the rows of that table.

14 · NESTED `IF`: GRADE *AND* PASS/FAIL

```
#include <iostream>
using namespace std;

int main() {
    int score;
    cout << "Enter student score: ";
    cin  >> score;

    if (score >= 50) {                    // outer: passed
        cout << "Status: PASS" << endl;
        if      (score >= 70) cout << "Grade: A" << endl;
        else if (score >= 60) cout << "Grade: B" << endl;
        else                  cout << "Grade: C" << endl;
    } else {                              // outer: failed
        cout << "Status: FAIL" << endl;
        if (score >= 45) cout << "Grade: D" << endl;
        else             cout << "Grade: F" << endl;
    }
    return 0;
}
```

This is what "nested" means: the outer `if` decides pass or fail, and a whole second decision lives *inside* each branch.

15 · LOGIN SYSTEM, 3 ATTEMPTS HIGH YIELD

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string username, password;
    bool   loggedIn = false;

    for (int attempt = 1; attempt <= 3; attempt++) {
        cout << "Attempt " << attempt << " of 3" << endl;
        cout << "Username: ";
        cin  >> username;
        cout << "Password: ";
        cin  >> password;

        if (username == "admin" && password == "secret") {
            loggedIn = true;
            break;                     // stop asking once correct
        }
        cout << "Incorrect. Try again.\n" << endl;
    }

    if (loggedIn) cout << "Login successful" << endl;
    else          cout << "Access denied"    << endl;
    return 0;
}
```

Three techniques in one answer — a counted `for` loop for the attempt limit, a **flag variable** (`loggedIn`) to remember what happened, and `break` to leave early. This is the most exam-shaped exercise in the whole document; learn it as a template.

12 · SUM NUMBERS UNTIL 0 IS ENTERED HIGH YIELD

```
#include <iostream>
using namespace std;

int main() {
    int num, sum = 0;      // sum MUST start at 0

    do {
        cout << "Enter a number (0 to stop): ";
        cin  >> num;
        sum += num;        // adding 0 changes nothing
    } while (num != 0);

    cout << "The total is: " << sum << endl;
    return 0;
}
```

The accumulator pattern: declare `sum = 0` **before** the loop, add inside it, print after. Forgetting to initialise `sum` is the classic bug — it then starts at whatever junk was in memory.

16 · FUNCTION THAT SQUARES A NUMBER

```
#include <iostream>
using namespace std;

int square(int n) {
    return n * n;
}

int main() {
    int num;
    cout << "Enter a number: ";
    cin  >> num;
    cout << "Square: " << square(num) << endl;
    return 0;
}
```

17 · CELSIUS TO FAHRENHEIT

```
#include <iostream>
using namespace std;

double toFahrenheit(double celsius) {
    return (celsius * 9.0 / 5.0) + 32;
}

int main() {
    cout << "0C   = " << toFahrenheit(0)   << "F" << endl;
    cout << "37C = " << toFahrenheit(37)  << "F" << endl;
    cout << "100C= " << toFahrenheit(100) << "F" << endl;
    return 0;
}
```

Write `9.0 / 5.0`, not `9 / 5` — the integer version gives 1 and every answer comes out wrong. Expected output: 32, 98.6, 212.

18 · BY VALUE *AND* BY REFERENCE IN ONE PROGRAM HIGH YIELD

```
#include <iostream>
using namespace std;

void doubleByValue(int x) {          // copy
    x = x * 2;
    cout << "  inside byValue:     " << x << endl;
}

void doubleByReference(int &x) {     // the actual variable
    x = x * 2;
    cout << "  inside byReference: " << x << endl;
}

int main() {
    int a = 10, b = 10;

    cout << "Before: a = " << a << ", b = " << b << endl;
    doubleByValue(a);
    doubleByReference(b);
    cout << "After:  a = " << a << ", b = " << b << endl;
    return 0;
}

OUTPUT
Before: a = 10, b = 10
  inside byValue:     20
  inside byReference: 20
After:  a = 10, b = 20      <-- only b changed
```

20 · GLOBAL AND LOCAL WITH THE SAME NAME

```
#include <iostream>
using namespace std;

int value = 100;                 // global

int main() {
    int value = 50;              // local, hides the global

    cout << "Local value:  " << value   << endl;   // 50
    cout << "Global value: " << ::value << endl;   // 100
    return 0;
}
```

The local variable "shadows" the global one. To reach the global anyway, use the **scope resolution operator `::`** — this is not in the course notes, so quoting it is free credit.

21 · STATIC LOCAL VARIABLE COUNTING CALLS NOT IN THE NOTES

```
#include <iostream>
using namespace std;

void countCalls() {
    static int count = 0;    // initialised ONCE, survives calls
    count++;
    cout << "This function has been called "
         << count << " time(s)." << endl;
}

int main() {
    countCalls();            // 1
    countCalls();            // 2
    countCalls();            // 3
    return 0;
}
```

The course document sets this exercise but never teaches `static`. A **static local variable** has the **scope of a local** — visible only inside the function — but the **lifetime of a global**: it is created once, keeps its value between calls, and is destroyed only when the program ends. Remove the word `static` and the counter resets to 0 every call, printing 1, 1, 1. That contrast is the whole answer.

22 · RECURSIVE SUM OF 1 TO N

```
#include <iostream>
using namespace std;

int sumTo(int n) {
    if (n <= 0) return 0;          // base case
    return n + sumTo(n - 1);       // recursive case
}

int main() {
    int n;
    cout << "Enter n: ";
    cin  >> n;
    cout << "Sum from 1 to " << n << " = " << sumTo(n) << endl;
    return 0;
}

TRACE, n = 5
sumTo(5) = 5 + sumTo(4)
sumTo(4) = 4 + sumTo(3)
sumTo(3) = 3 + sumTo(2)
sumTo(2) = 2 + sumTo(1)
sumTo(1) = 1 + sumTo(0) = 1 + 0
         → 1, 3, 6, 10, 15
```

19 · SWAP TWO NUMBERS BY REFERENCE

```
#include <iostream>
using namespace std;

void swapNumbers(int &x, int &y) {
    int temp = x;      // the third variable is essential
    x = y;
    y = temp;
}

int main() {
    int a = 5, b = 9;
    cout << "Before swap: a = " << a << ", b = " << b << endl;
    swapNumbers(a, b);
    cout << "After swap:  a = " << a << ", b = " << b << endl;
    return 0;
}
```

A swap only works by reference — by value it would swap two copies and the caller would see nothing. Say that sentence in the answer; it is the point of the exercise.

23 · RECURSIVE FIBONACCI, N TERMS HIGH YIELD

```
#include <iostream>
using namespace std;

int fibonacci(int n) {
    if (n == 0) return 0;                  // base case 1
    if (n == 1) return 1;                  // base case 2
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int terms;
    cout << "How many terms? ";
    cin  >> terms;

    cout << "Fibonacci sequence: ";
    for (int i = 0; i < terms; i++) {
        cout << fibonacci(i) << " ";
    }
    cout << endl;
    return 0;
}

For terms = 8:   0 1 1 2 3 5 8 13
```

Fibonacci needs two base cases, because the recursive step reaches back two places. Add the remark that this naive version **recomputes the same values many times** — a point worth a mark if the question asks about efficiency.

3 Trace drills — predict the output

Cover the right-hand column. "What does this program print?" is the cheapest question to set and the easiest to get wrong in a hurry.

#	CODE	OUTPUT	WHY

1	`int a = 10, b = 3;`
	`cout << a / b << endl;`
	`cout << a % b << endl;`
		3
		1
			Integer division **truncates**; `%` gives the remainder.

2	`for (int i = 0; i < 3; i++)` `    cout << i << " ";`	**0 1 2**	Starts at 0 and stops **before** 3 — three iterations, not four.
3	`int i = 5;` `while (i > 0) { cout << i << " "; i -= 2; }`	**5 3 1**	Stops when `i` reaches −1, which fails the test.
4	`int x = 0;` `do { cout << "run "; } while (x != 0);`	**run**	`do-while` tests **after** the body, so it always runs **at least once** even though the condition is false.

5	`int n = 2;`
	`switch (n) {`
	`  case 1: cout << "one ";`
	`  case 2: cout << "two ";`
	`  case 3: cout << "three ";`
	`}`
		two three	**No `break;`** — execution **falls through** from case 2 into case 3.

6	`for (int i = 1; i <= 2; i++)`
	`  for (int j = 1; j <= 3; j++)`
	`    cout << i * j << " ";`
		1 2 3 2 4 6	The inner loop runs fully for **each** outer value — 2 × 3 = 6 numbers.

7	`void f(int x) { x = 99; }`
	`int main() { int a = 1; f(a);`
	`             cout << a; }`
		1	Passed **by value** — the function changed only its own copy.

8	`void f(int &x) { x = 99; }`
	`int main() { int a = 1; f(a);`
	`             cout << a; }`
		99	Passed **by reference** — the original variable was modified.

9	`int f(int n) {`
	`  if (n == 0) return 1;`
	`  return n * f(n - 1); }`
	`cout << f(4);`
		24	4 × 3 × 2 × 1 × 1 — factorial by recursion.

10	`int x = 5;` `cout << (x > 3) << " " << (x < 3);`	**1 0**	A relational expression is a `bool`, printed as **1** for true and **0** for false.

11	`void g() { static int c = 0; c++;`
	`           cout << c << " "; }`
	`g(); g(); g();`
		1 2 3	`static` is initialised **once** and keeps its value between calls. Without it: **1 1 1**.

12	`int i;`
	`for (i = 1; i <= 5; i++) { }`
	`cout << i;`
		6	The loop exits only **after** the update makes the test fail — a classic off-by-one.

4 Spot the error — find the bug and name it

#	FAULTY CODE	ERROR TYPE	FIX
1	`cin << number;`	Syntax	Arrows the wrong way: `cin >> number;`
2	`if (x = 5) cout << "five";`	**Logic**	`=` assigns and is always true; use `if (x == 5)`.
3	`for (int i = 0; i < 5;) cout << i;`	Runtime	No update — infinite loop. Add `i++`.
4	`int f(int n){ return n * f(n-1); }`	Runtime	**No base case** — infinite recursion, stack overflow. Add `if (n == 0) return 1;`
5	`string s = 'hello';`	Syntax	Single quotes are for one `char`. Use `"hello"`.
6	`cout << "Average: " << total / 4;` with `int total`	**Logic**	Integer division truncates the average. Use `total / 4.0`.
7	A `switch` whose cases have no `break;`	**Logic**	Fall-through runs every later case. Add `break;` to each.
8	Using `string` with only `#include <iostream>`	Syntax	Add `#include <string>`.
9	`void swap(int x, int y){ int t=x; x=y; y=t; }`	**Logic**	Swaps copies only. Declare as `(int &x, int &y)`.

#	FAULTY CODE	ERROR TYPE	FIX
10	Reading `localVar` in `main()` when it was declared in another function	Syntax	Out of **scope**. Declare it in `main()`, make it global, or return it from the function.

Name the three error types if asked: syntax errors break the language rules and are caught by the compiler · **logic errors** compile and run but produce the wrong answer · **runtime errors** compile but fail during execution, e.g. division by zero or infinite recursion. **Logic errors are the most dangerous because nothing warns you.**

5 Past-style questions, fully worked SUPPLIED

Q1 (a) · What is structured programming and what is its importance?

Definition — open with this. Structured programming is a programming approach that emphasizes the use of **clear control structures** — sequence, selection and iteration — together with **modular design**, instead of arbitrary jumps such as the `goto` statement. Its philosophy is to **break a complex problem into smaller, manageable parts and solve each part systematically.** It emerged as a response to the tangled, unmaintainable "**spaghetti code**" produced by unstructured programming.

The three permitted constructs — name them, they frame the whole answer:

CONSTRUCT	MEANING	IN C++
Sequence	Statements executed one after another, in order	Ordinary statements
Selection	A choice between alternative paths	`if`, `if-else`, `switch`
Iteration	Repetition of a block while a condition holds	`for`, `while`, `do-while`

> **A line that lifts the answer:** every program, however complex, can be written using only these three constructs — which is why `goto` is unnecessary, and why banning it costs nothing and buys clarity.

IMPORTANCE OF STRUCTURED PROGRAMMING — WRITE NINE POINTS

1. **Improved readability and clarity.** The logical flow mirrors human reasoning, so code can be read top to bottom rather than traced through jumps.
2. **Easier debugging and testing.** Errors are **isolated within modules**, so a fault can be located and fixed without reading the whole program.
3. **Easier maintenance and modification.** Programs can be extended gracefully over time instead of being rewritten.
4. **Modularity and code reuse.** A function written once can be used in many places, reducing redundancy.
5. **Reduced development time and cost**, because reusable, well-organised code is written once and tested once.
6. **Supports teamwork.** Different programmers can work on separate modules without interfering with each other.
7. **Manages complexity** through divide-and-conquer — large problems become sets of small, solvable ones.
8. **Greater reliability.** Predictable control flow means fewer of the unpredictable-jump bugs that plague unstructured code.
9. **A foundation for advanced paradigms.** Mastering structured programming is **essential before moving on to object-oriented programming** and larger software projects.

Close with the notes' own sentence: **structured programming replaces chaos with order**, ensuring programs are not only functional but maintainable in the long run.

Q1 (b) · Determine the value of the following

Given **a = −3, b = 5, c = −2, d = 6**. Apply **precedence** first — `*`, `/` and `%` bind tighter than `+` and `-` — then work left to right. **Show every step;** the marks are for the working, not the number.

(i) `a - b + c / 2 - d * 5`

```
Step 1 — do / and * first
        c / 2  = -2 / 2 = -1
        d * 5  =  6 * 5 = 30

Step 2 — substitute
        = a - b + (-1) - (30)
        = -3 - 5 + (-1) - 30

Step 3 — left to right
        -3 - 5   = -8
        -8 + (-1)= -9
        -9 - 30  = -39

                    ANSWER = -39
```

(ii) `a - c + d / 2 + a² - 10`

```
Step 1 — powers and division first
        d / 2   = 6 / 2 = 3
        a²      = (-3)² = 9       ← square of a NEGATIVE
                                    number is POSITIVE

Step 2 — substitute
        = -3 - (-2) + 3 + 9 - 10

Step 3 — left to right
        -3 - (-2) = -3 + 2 = -1
        -1 + 3             =  2
         2 + 9             = 11
        11 - 10            =  1

                    ANSWER = 1
```

TWO TRAPS IN THAT PAIR

1. `a - (-2)` **is** `a + 2`. Subtracting a negative adds. This single sign is where most of the lost marks in (ii) come from.
2. `(-3)²` **= +9, not −9** — squaring always gives a positive result.

And one for the C++ purists. In real C++ the symbol `^` is **not** "to the power of" — it is the **bitwise XOR** operator, and it binds **more loosely than** `+` **and** `-`. So if `a^2` were typed literally into a program, the compiler would read the whole line as `(a - c + d/2 + a) ^ (2 - 10)` = `(-1) ^ (-8)` = 7, not 1. **Answer the arithmetic question as intended — a² = 9, giving 1 — but naming this discrepancy in one line shows genuine command of the language.** To raise a number to a power in C++ you use `pow(a, 2)` from `<cmath>`, or simply `a * a`.

Q2 · Trace the output from the following code

```
int main() {
    int sum = 0;
    for (int i = 1; i <= 50; i++) {
        sum += i;
        cout << i << " ";
    }
    cout << "\nSum of primes between 1 and 50 = " << sum
         << endl;
    return 0;
}
```

THE TRACE

ITERATION	I	SUM += I	PRINTED
1	1	0 + 1 = 1	1
2	2	1 + 2 = 3	2
3	3	3 + 3 = 6	3
4	4	6 + 4 = 10	4
…	…	…	…
50	50	1225 + 50 = **1275**	50

The loop runs **50 times**, printing every integer from 1 to 50 and accumulating each into `sum`. The total is the sum of the first 50 natural numbers:

$$n(n + 1) / 2 = 50 \times 51 / 2 = \mathbf{1275}$$

THE EXACT OUTPUT

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
34 35 36 37 38 39 40 41 42 43 44 45 46 47 48
49 50
Sum of primes between 1 and 50 = 1275
```

(All fifty numbers appear on **one line** separated by spaces — they are wrapped above only to fit the page. The `\n` at the start of the last `cout` is what moves to a new line before the message.)

THE POINT OF THE QUESTION — SAY THIS

The program's **message is a lie**. It claims to print the **sum of primes**, but the loop adds **every integer** from 1 to 50 and never tests primality at all. The printed value **1275** is the sum of 1…50; the true sum of the primes between 1 and 50 is **328**.

This is a **logic error**, not a syntax error: the code compiles and runs perfectly, which is exactly why logic errors are the most dangerous kind. It is also a **readability failure** of the sort this course warns about — the output label does not describe what the code does.

THE CORRECTED PROGRAM, IF ASKED TO FIX IT

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n < 2) return false;             // 0 and 1 are not prime
    for (int k = 2; k * k <= n; k++) {
        if (n % k == 0) return false;    // a divisor found
    }
    return true;
}

int main() {
    int sum = 0;
    for (int i = 1; i <= 50; i++) {
        if (isPrime(i)) {
            sum += i;
            cout << i << " ";
        }
    }
    cout << "\nSum of primes between 1 and 50 = "
          << sum << endl;
    return 0;
}

OUTPUT
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
Sum of primes between 1 and 50 = 328
```

Note the modularity: the primality test is its own function, so `main()` stays readable — the exact principle Module 1 teaches, demonstrated in the fix.

6 Mock paper — sit this closed-book

RIVERS STATE UNIVERSITY · PGD COMPUTER SCIENCE · CMS 708 — STRUCTURED PROGRAMMING
Mock examination · Time allowed: 3 hours · **Answer any FIVE questions** · Each question carries 20 marks · All code must be complete and compilable

Q	QUESTION	MARKS
1	**(a)** Distinguish between structured and unstructured programming, using a table with at least five bases of comparison. **(b)** Write two short C++ programs that perform the same task — one using `goto` and one using a loop — and explain which is preferable and why. **(c)** What is spaghetti code?	8 + 9 + 3
2	**(a)** Explain **modularity** and **readability** and state why each matters. **(b)** Write a program that reads two numbers and prints their sum, difference, product and quotient **using a separate function for each operation**. **(c)** Why is C++ described as a structured programming language? Give three reasons.	6 + 8 + 6
3	**(a)** List the six common C++ data types and give an example value of each. **(b)** State the five categories of operator in C++ with two examples each. **(c)** Write a payroll program that reads an employee's name, gross salary and tax rate, then computes and prints the net salary.	6 + 5 + 9
4	**(a)** Differentiate between the `for`, `while` and `do-while` loops, stating when each should be used. **(b)** Write a program using a `switch` statement to simulate an ATM menu. **(c)** Write a program that prints a multiplication table from 1 to 5 using nested `for` loops, and state how many lines it prints.	6 + 7 + 7
5	**(a)** Differentiate between passing parameters **by value** and **by reference**, using a table and a short program that demonstrates both. **(b)** Write a function that swaps two numbers, and explain why it must use one of the two methods. **(c)** Differentiate between the **scope** and the **lifetime** of a variable.	9 + 6 + 5
6	**(a)** What is recursion? State the two things every recursive function must have. **(b)** Write a recursive function to compute the factorial of a number and trace its execution for n = 5. **(c)** Write a recursive function to generate the Fibonacci sequence up to n terms.	5 + 8 + 7
7	**(a)** Define modularisation and state three benefits, illustrating with a real-world analogy. **(b)** Compare **top-down** and **bottom-up** program design using a table of at least four aspects, with a C++ illustration of each. **(c)** Which would you use in practice, and why?	6 + 10 + 4

Where every answer lives. Q1 → Vol I §2 · Q2 → Vol I §2 and exercise 4 · Q3 → Vol I §3 · Q4 → Vol I §4 and exercises 9 and 13 · Q5 → Vol I §5 and exercises 18 and 19 · Q6 → Vol I §5 and exercises 22 and 23 · Q7 → Vol I §6. **Every one of the seven is answerable from Volume I plus this pack** — which tells you how tightly this course's material maps onto its likely paper.

7 Walk-in sheet — the last ten minutes

THE PROGRAM SKELETON — WRITE THIS FIRST, EVERY TIME

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // declarations
    // prompt and cin
    // process
    // cout
    return 0;
}
```

THE TEN FACTS MOST LIKELY TO BE TESTED

1. **Structured** = clear control structures + modularity; **unstructured** = `goto` + spaghetti code
2. **Two pillars:** modularity (independent reusable units) and readability (understandable to others)
3. **Data types:** int · float · double · char · bool · string
4. **Operators:** arithmetic · relational · logical · assignment · increment/decrement
5. **Selection:** `if` · `if-else` · `switch` (needs `break;` and `default:`)
6. **Loops:** `for` = known count · `while` = unknown · `do-while` = **at least once**
7. **By value** = copy, original unchanged · **by reference** (`&`) = original modified
8. **Scope** = where accessible · **lifetime** = how long it exists
9. **Recursion** = a function calling itself; always needs a **base case**
10. **Top-down** = big picture, stepwise refinement (house) · **bottom-up** = building blocks, reusability (car)

THE FOUR ANALOGIES — ONE SENTENCE EACH, FREE MARKS

- **Unstructured code** = a traffic system where cars teleport to random roads
- **Modularisation** = planning a wedding by dividing catering, decoration, music, photography, guests
- **Top-down** = an architect designing a house, then plumbing and wiring
- **Bottom-up** = engineers building an engine and tyres, then assembling the car

FIVE REFLEXES THAT SAVE MARKS

1. Headers and `return 0;` on every program, no exceptions.
2. `cout <<` out, `cin >>` in.
3. `==` compares, `=` assigns.
4. Want a decimal? Make one operand a `double` — `9.0 / 5.0`, `total / 4.0`.
5. Indent one level per brace. On this course, **readability is itself on the syllabus.**

IF YOUR MIND GOES BLANK

Write the **skeleton**, then the **variable declarations** the question implies, then the **prompts and `cin`**. By the time those are on the page the logic in the middle is usually obvious — and even if it is not, most of the structure marks are already earned.